Chapter4 Dynamic Programming

核心前提： 动态规划算法假设我们拥有一个**完美的环境模型**（即我们完全知道环境的概率分布 $p(s', r|s, a)$）。它的目标是计算最优策略 π^* 和最优价值函数 V^*。

第一部分：策略评估 (Policy Evaluation)

—— 这里的任务是：给定一个策略，算出它到底有多好。

1. 直觉与生活例子

想象你是个围棋新手，你手里有一本棋谱（策略 π）。你想知道按照这本棋谱下棋，在不同的盘面（状态 s）下，你最终赢棋的概率（价值 V）是多少。

你不需要真的去下棋，因为你知道规则（环境模型）。你可以推演："如果我这步走A，对手可能走B或C...如果走B我就..."。通过这种**推演**，你可以算出当前局面的价值。

2. 数学定义：贝尔曼方程 (The Bellman Equation)

我们要计算的是**状态价值函数** $V^\pi(s)$。它必须满足贝尔曼方程：

$$V^\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{s', r} p(s', r|s, a) \left(r + \gamma V^\pi(s')\right)$$

符号详细拆解：

- $\pi(a|s)$：**策略**。在当前状态 s 决定做动作 a 的概率。
- $p(s', r|s, a)$：**环境动力学**。做了动作 a 后，环境把你送到 s' 并给你奖励 r 的概率。
- r：**即时奖励**。眼前马上拿到的好处。
- $\gamma V^\pi(s')$：**未来折扣价值**。$V^\pi(s')$ 是下一站的价值，γ (0~1) 是这一站价值对现在的折损率。

3. 如何计算：迭代法

直接解方程很难，我们用"猜"和"更新"的方法：

1. **初始化**：把所有状态的价值 $V_0(s)$ 随便设（比如都设为0）。
2. **扫描更新**：利用 V_k 算出 V_{k+1}。

$V_{k+1}(s) \leftarrow \mathbb{E}_\pi[r + \gamma V_k(s')]$

3. **停止**：当 V_{k+1} 和 V_k 几乎没区别时，我们就得到了真实的 V^π。

实现细节：

- **并行更新 (Parallel)**：用两个数组，旧表算完填入新表。
- **原地更新 (In-place)**：只用一个数组，算出新的直接覆盖旧的。**原地更新通常收敛更快**，因为后面的状态能立刻用到前面刚更新的数据。

第二部分：策略改进 (Policy Improvement)

—— 这里的任务是：既然知道现在的策略表现如何，怎么让它变好？

1. 直觉与生活例子

你每天按固定路线（旧策略 π）上班。通过评估，你知道这大概花30分钟。

今天你在某个路口（状态 s）想："如果我尝试拐进这条没走过的巷子（动作 $a \neq \pi(s)$），之后再按老规矩走，会不会更快？"

如果你发现走巷子只需要28分钟，那么恭喜你，你找到了一个**改进**。以后在这个路口，你应该永远选择走巷子。

2. 数学原理：策略改进定理

我们引入 **动作价值函数** $Q^\pi(s,a)$，它代表"在 s 强行做动作 a，之后按 π 走"的价值。

定理核心：

如果在任何状态 s 下，有一个新动作 a 使得 $Q^\pi(s,a) > V^\pi(s)$，那么在这个状态选 a 就会生成一个更好的策略 π'。

$$
\begin{aligned}
V^\pi(s) \leq Q^\pi(s,\pi'(s)) &= \mathbb{E}\left[R_{t+1} + \gamma V^\pi(S_{t+1}) \mid S_t = s, A_t = \pi'(s)\right] \\
&= \mathbb{E}_{\pi'}\left[R_{t+1} + \gamma V^\pi(S_{t+1}) \mid S_t = s\right] \\
&\leq \mathbb{E}_{\pi'}\left[R_{t+1} + \gamma Q^\pi(S_{t+1}, \pi'(S_{t+1})) \mid S_t = s\right] \\
&= \mathbb{E}_{\pi'}\left[R_{t+1} + \gamma \mathbb{E}_{\pi'}\left[R_{t+2} + \gamma V^\pi(S_{t+2}) \mid S_{t+1}\right] \mid S_t = s\right] \\
&= \mathbb{E}_{\pi'}\left[R_{t+1} + \gamma R_{t+2} + \gamma^2 V^\pi(S_{t+2}) \mid S_t = s\right].
\end{aligned}
$$

推导简述：

$$V^\pi(s) \le Q^\pi(s,\pi'(s)) = \mathbb{E}[R_{t+1} + \gamma V^\pi(S_{t+1})] \le \mathbb{E}[R_{t+1} + \gamma Q^\pi(S_{t+1},\pi'(S_{t+1}))] \cdots = V^{\pi'}(s)$$

这就像滚雪球，每一步都好一点点，累积起来整个策略就变好了。

3. 构造新策略：贪婪法 (Greedy)

我们不需要一个个试，直接让新策略 π' 在每一步都选**最**好的那个动作：

$$\pi'(s) \doteq \arg\max_a Q^\pi(s,a)$$

这就叫**贪婪策略**。根据定理，贪婪策略一定优于或等于旧策略。

第三部分：策略迭代 (Policy Iteration)

—— 这里的任务是：把“评估”和“改进”连成一个圈，直到找到最优解。

Algorithm 4.1 Policy iteration algorithm

```
for each s ∈ S do                                          ▷ Initialization
    Initialize π(s) ∈ A
    Initialize V(s) ∈ ℝ
end for
repeat
    repeat                                                 ▷ Policy evaluation
        Let Δ ← 0
        for each s ∈ S do
            Let v' ← V(s)
            Update V(s) ← Σ_{s',r} p(s',r|s,π(s)) [r + γV(s')]
            Let Δ ← max(Δ, |v' − V(s)|)
        end for
    until Δ < ϵ
    Let pstable ← true
    for each s ∈ S do                                      ▷ Policy improvement
        for each a ∈ A do
            Let Q(a) = Σ_{s',r} p(s',r|s,π(s)) [r + γV(s')]
        end for
        Let a' ← argmax_a and q' ← Q(a')
        if q' > Q(π(s)) then
            Update π(s) ← a'
            Let pstable ← false
        end if
    end for
until pstable = true
return π_* ≃ π and V* = v
```

1. 核心流程

这是一个循环过程：

$$\pi_0 \xrightarrow{\text{评估}} V^{\pi_0} \xrightarrow{\text{改进}} \pi_1 \xrightarrow{\text{评估}} V^{\pi_1} \cdots \longrightarrow \pi_* \longrightarrow V^*$$

1. **策略评估 (E)**：把当前策略 π 的价值 V 算得准准的。
2. **策略改进 (I)**：根据 V 找到更好的 π'。
3. **终止条件**：如果某次改进后，策略不再变化（$stable = true$），说明找到了最优策略 π^*。

2. 关键性质

因为有限状态的马尔可夫决策过程（MDP）只有有限种可能的策略，而且每次改进策略都会变好，所以**策略迭代一定会在有限步内收敛到最优**。

第四部分：值迭代 (Value Iteration)

—— 这里的任务是：策略评估太慢了，能不能搞快点？

1. 问题与思路

策略迭代里的“评估”步骤很费劲，每次都要循环很多遍直到收敛。

值迭代的想法：我们不需要把 V 算得那么准。我们只更新一轮 V，甚至不关心策略长什么样，直接把“改进”融合进“评估”里。

2. 核心公式：贝尔曼最优方程迭代

我们不再用普通的贝尔曼方程，而是直接迭代**贝尔曼最优方程**：

$V_{k+1}(s) \leftarrow \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma V_k(s')]$

注意这里是 $\max_a$（找最大值），而不是 $\sum \pi(a|s)$（求平均）。

这相当于：每一步更新价值时，直接假定我们会选最好的那个动作。

第五部分：广义策略迭代 (GPI)

—— 这里的任务是：理解所有 RL 算法的本质。

1. 概念

无论是策略迭代、值迭代，还是后面会学到的异步 DP，它们都包含两个互相作用的过程：

1. **评估 (Evaluation)**：让 V 变得符合当前的 π。
2. **改进 (Improvement)**：让 π 对当前的 V 变得贪婪。

2. 互动图解

这两个过程就像在互相追逐：策略变了，价值函数就不准了；价值函数变了，原来的策略就不是最优了。它们互相博弈，直到达到一个平衡点——**最优解**。

3. 效率与异步 (Asynchronous DP)

- **效率**：DP 算法找到最优解的时间是多项式级别的，比直接暴力搜索策略快得多。
- **异步 DP**：为了处理大状态空间，我们不需要每次都按顺序扫描所有状态。我们可以随机选状态更新，或者只更新重要的状态。只要所有状态都能被访问到，算法最终也能收敛。

---

1. 核心概念结构

- **策略评估 (Prediction)**：V^π 是多少？(输入 π, 输出 V)
- **策略改进 (Control)**：如何利用 V 获得更好的 π'？
- **策略迭代** = 评估(完全收敛) + 改进
- **值迭代** = 评估(一步) + 改进 (融合在一起)

2. 关键公式速查

名称	公式	含义
贝尔曼方程	$V^\pi(s) = \sum \pi \sum p(r + \gamma V^\pi(s'))$	当前策略下的价值平衡
贝尔曼最优方程	$V^*(s) = \max_a \sum p(r + \gamma V^*(s'))$	最优策略下的价值平衡
Q值公式	$Q^\pi(s,a) = \sum p(r + \gamma V^\pi(s'))$	动作价值：这一步做a，以后按π做
贪婪策略	$\pi'(s) = \arg\max_a Q(s,a)$	只看眼前的最佳选择

3. 常见误区澄清

- **误区**：值迭代最后只得到 V^*，没有策略。
  - **纠正**：有了 V^*，我们可以通过一步贪婪搜索 ($\arg\max_a$) 轻松提取出 π^*。
- **误区**：一定要把 V 算得完全精确才能改进策略。

- **纠正**：不需要。**广义策略迭代 (GPI)** 告诉我们，哪怕 V 只是大概准，做出的改进通常也是有效的。值迭代就是极端的例子（只算一步就改）。

4. 与其他概念的联系

- **与强化学习的关系**：DP 是 RL 的理论基础，但 DP 需要知道 $p(s'|s,a)$（模型）。真正的 RL（如 Q-Learning）是在不知道 p 的情况下，通过采样来近似这些 DP 公式。
- **与 Unity 智能体的联系**：如果你的 Unity 环境很简单（格子世界），你可以直接写 DP 代码算出最优路径。如果环境复杂，DP 的思想（评估+改进）依然是训练深度强化学习 (Deep RL) 智能体的核心逻辑。